

Software Testing & Quality Assurance (ST&QA)

Unit 1: Introduction to Software Testing

Sonali Bhoite

Course Objectives

- Introduction to basic concepts of software testing
- Introduction to basic concepts of software quality.
- To know about test planning and management.
- To understand automation testing.
- To know details about specialized testing.
- To study software quality tools.

Syllabus

- **Unit I- Introduction to Software Testing**

- **Overview of Software Testing** : Definition and importance, Software Development Life Cycle (SDLC) and Testing Life Cycle (STLC),
- **Types of Testing**: Manual Testing, Automated Testing,
- **Testing Levels** : Unit Testing, Integration Testing, System Testing, Acceptance Testing,
- **Static Testing Techniques**: Reviews, Walkthroughs, Inspections,
- **Dynamic Testing Techniques**: **Black Box Testing**: Equivalence, Partitioning, Boundary Value Analysis, Decision Table Testing, **White Box Testing**: Statement, Coverage, Branch Coverage, Path Coverage.

What is Software Testing?

- Software testing is the process of **evaluating a software system or its components** to find whether it meets specified requirements and to identify defects.
- It's about **verification (Are we building the product right?)** and **validation (Are we building the right product?)**

Why is Testing Important?

- Ensures software quality and reliability
- Helps in finding and fixing bugs early
- Reduces cost of rework
- Builds customer confidence
- Prevents catastrophic failures in critical systems (e.g., healthcare, aviation)

Goals of Software Testing

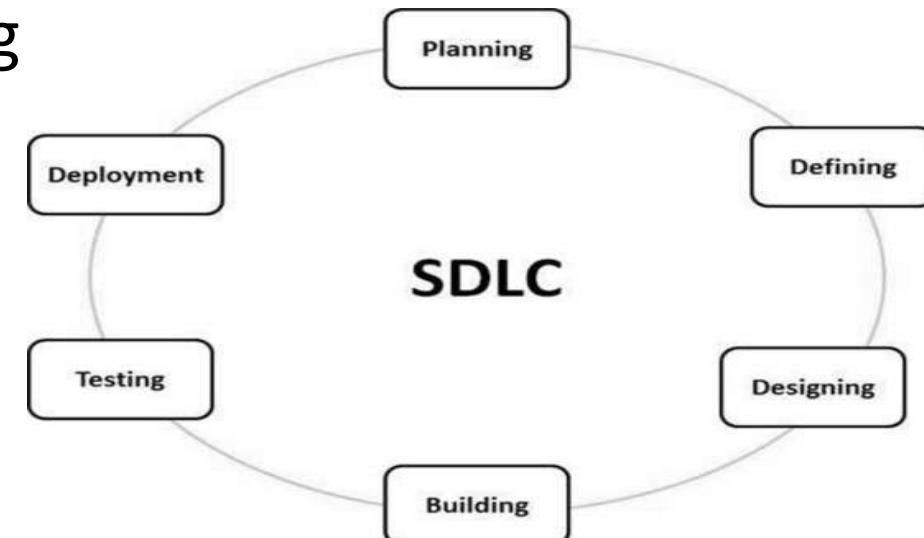
- **Detect faults or errors** that might have been introduced during development
- Ensure the software **meets user requirements**
- Validate that the software is **fit for purpose**
- Ensure **robustness, reliability, and security**

Key Terminology

Error	A human mistake in coding or design
Defect	A flaw in the software due to an error
Failure	Deviation of software from its expected behavior during execution
Bug	Informal term for defect
Test Case	A set of conditions under which a tester determines if software works correctly

Software Development Life Cycle (SDLC) & Testing

- Testing is integrated at different stages of the SDLC:
- **Requirement Phase** → Static Testing (e.g., Reviews)
- **Design Phase** → Design Verification
- **Implementation Phase** → Unit Testing
- **After Integration** → Integration Testing
- **Before Release** → System & Acceptance Testing



Principles of Software Testing

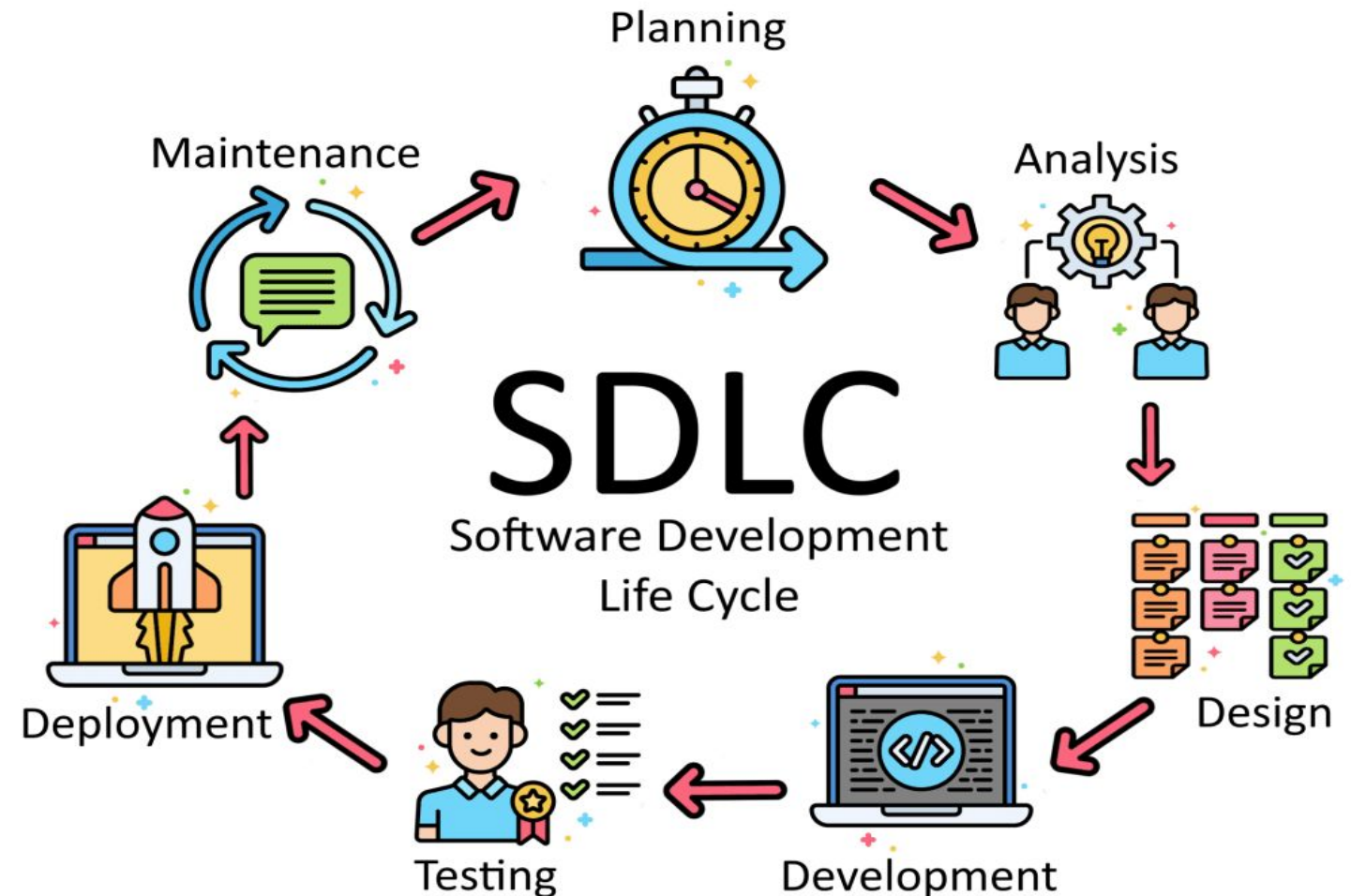
- Testing shows the presence of defects, not their absence
- Exhaustive testing is impossible
- Early testing saves time and money
- Defect clustering (most bugs in few modules)
- Pesticide paradox (same tests find fewer new bugs)
- Testing is context dependent
- Absence of errors $\neq$ Useful software

Software Development Life Cycle (SDLC)

SDLC is a **structured process** followed to build software in a **systematic and efficient** manner.

It defines **phases** for:

- Planning
- Designing
- Developing
- Testing
- Deploying
- Maintaining software





Software Development Life Cycle (SDLC)

1. Requirement Gathering & Analysis

- Identify stakeholders
- Collect functional & non-functional requirements
- Document and analyze user needs
- Feasibility study (technical, economic, legal)

2. System Design

- Design system architecture
- Data models, database design
- User Interface (UI) design
- Technology stack selection
- Security design planning

Software Development Life Cycle (SDLC)

3. Implementation (Coding)

- Break down modules
- Assign tasks to developers
- Code as per design
- Use version control (e.g., GitHub)

4. Testing

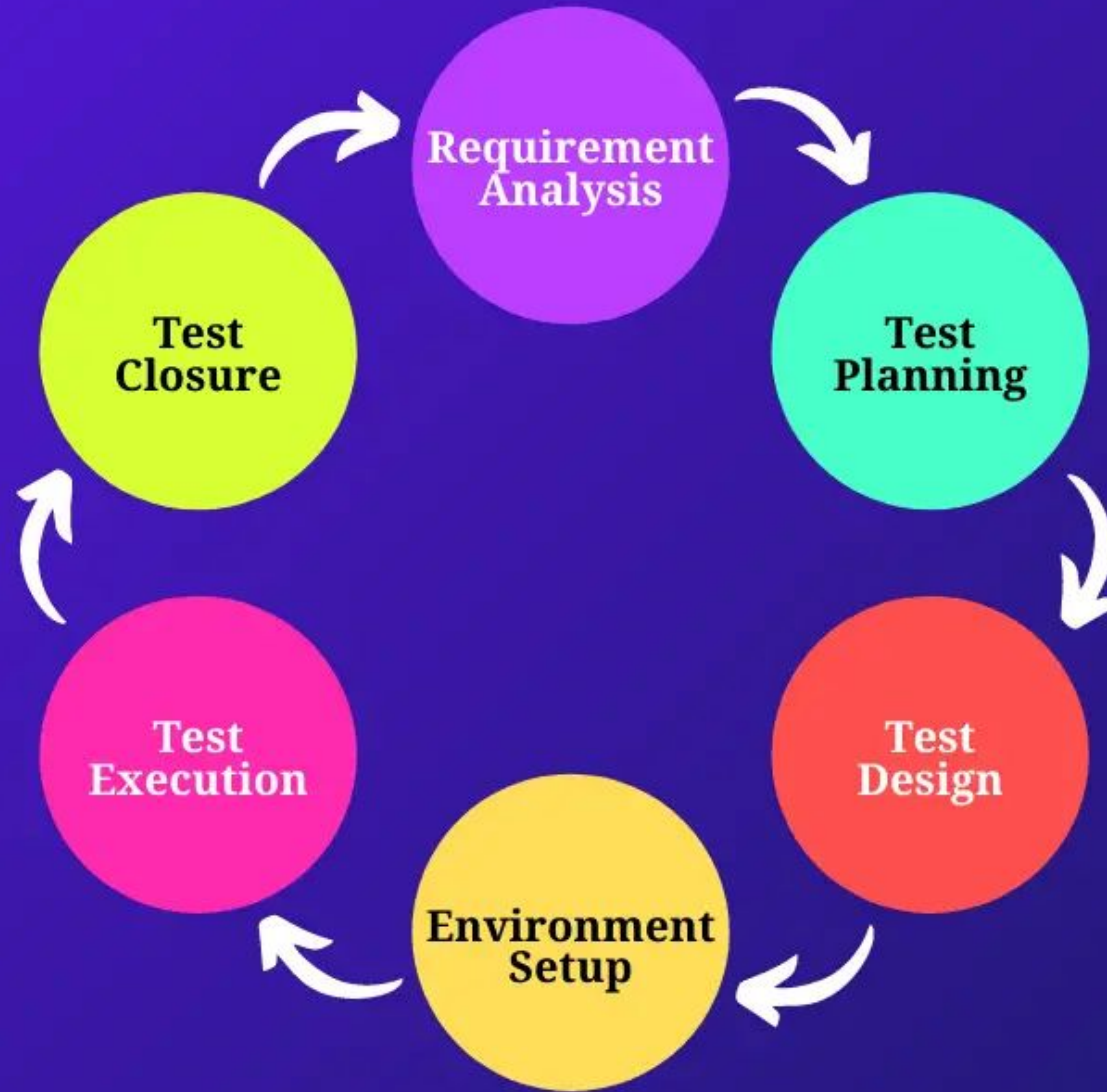
- Unit testing (by developers)
- Integration testing
- System testing
- User Acceptance Testing (UAT)
- Bug fixing

5. Deployment

- Prepare production environment
- Deploy to server/cloud
- Monitor for runtime errors
- Release to users

6. Maintenance

- Fix bugs after release
- Add new features
- Improve performance
- Handle software upgrades



SOFTWARE TESTING LIFE CYCLE

Software Testing Life Cycle (STLC)

•1. Requirement Analysis

Goal: Understand what needs to be tested.

Activities

- Study SRS, BRD, user stories
- Identify testable requirements
- Prepare Requirement Traceability Matrix (RTM)
- Identify test types (functional, performance, etc.)

Deliverables

- RTM
- Requirement analysis report

2. Test Planning

Goal: Define the overall testing strategy.

Activities

- Decide scope, objectives, resources
- Select tools and environments
- Estimate effort and schedule
- Identify risks and mitigation

Deliverables

- Test Plan
- Effort estimation
- Test schedule

Software Testing Life Cycle (STLC)

•3. Test Case Development

Goal: Design tests that validate requirements.

Activities

- Write test cases & scenarios
- Prepare test data
- Review & baseline test cases

Deliverables

- Test cases
- Test data
- Test scripts (if automation)

4. Test Environment Setup

Goal: Prepare a realistic testing setup.

Activities

- Configure hardware/software
- Deploy builds
- Smoke test environment readiness

Deliverables

- Environment readiness report

Software Testing Life Cycle (STLC)

5. Test Execution

Goal: Run tests and find defects.

Activities

- Execute test cases
- Log defects
- Re-test fixes
- Perform regression testing

Deliverables

- Execution reports
- Defect reports

6. Test Closure

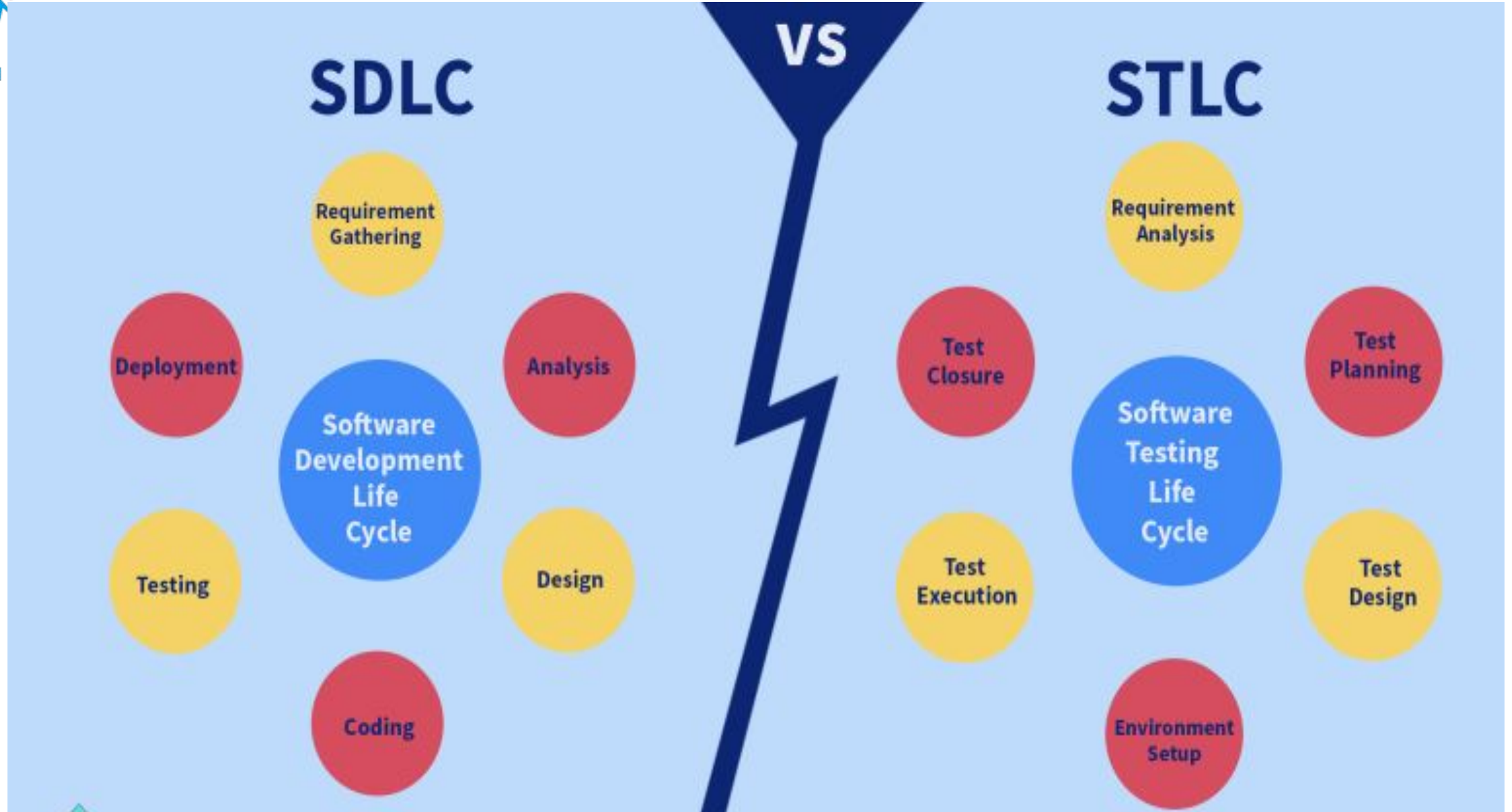
Goal: Wrap up testing and capture learnings.

Activities

- Verify exit criteria
- Prepare test summary
- Analyze defect metrics
- Document lessons learned

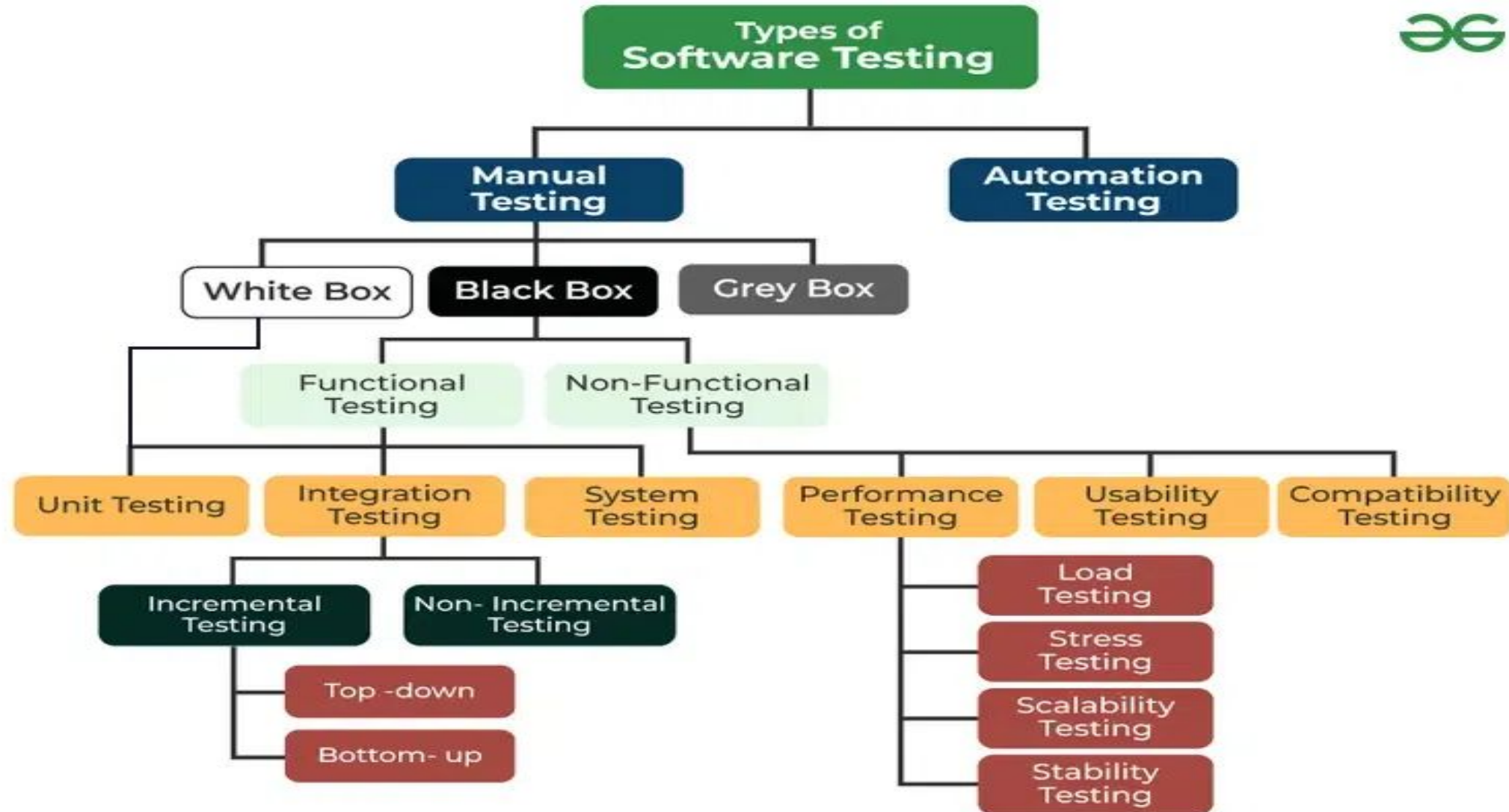
Deliverables

- Test closure report
- Metrics & sign-off



	SDLC	STLC
 Core Objective	Oversees the complete development of a software product	Focuses specifically on ensuring the software is tested thoroughly.
 Phases Involved	Encompasses planning, designing, building, testing, and maintenance.	Consists of test planning, test case development, environment setup, and test
 Team Composition	Includes project managers, business analysts, developers, and designers.	Involves quality assurance personnel and testers who specialize in identifying defects.
 Outcome	Includes project managers, business analysts, developers, and designers.	Involves quality assurance personnel and testers who specialize in identifying defects.
 Primary Focus	Prioritizes developing functional and feature-complete software	Emphasizes ensuring software is defect-free and meets quality standards

Types of Testing: Manual Testing, Automated Testing



Types of Testing: Manual Testing, Automated Testing

Manual Testing

Definition

- Manual testing is the process where **testers execute test cases manually** without using any automation tools to verify software functionality.

Features

- Human intervention required
- Test cases executed step-by-step by testers
- Suitable for exploratory and usability testing



Types of Testing: Manual Testing, Automated Testing

Advantages

- No programming knowledge required
- Useful for UI/UX and ad-hoc testing
- Flexible for small projects

Disadvantages

- Time-consuming
- Error-prone (human mistakes)
- Not suitable for repetitive testing

Example

- A tester manually checks:
 - Login page
 - Form submission
 - Navigation links



Types of Testing: Manual Testing, Automated Testing

Automated Testing

Definition: Automated testing uses **software tools and scripts** to execute test cases automatically and compare expected vs actual results.

Features

- Uses tools like Selenium, JUnit, TestNG
- Faster execution
- Reusable test scripts

Advantages

- Saves time for repetitive tasks
- High accuracy
- Suitable for regression testing
- Supports continuous integration

Types of Testing: Manual Testing, Automated Testing

Disadvantages

- Requires technical skills
- Initial setup cost is high
- Not ideal for usability testing

Example

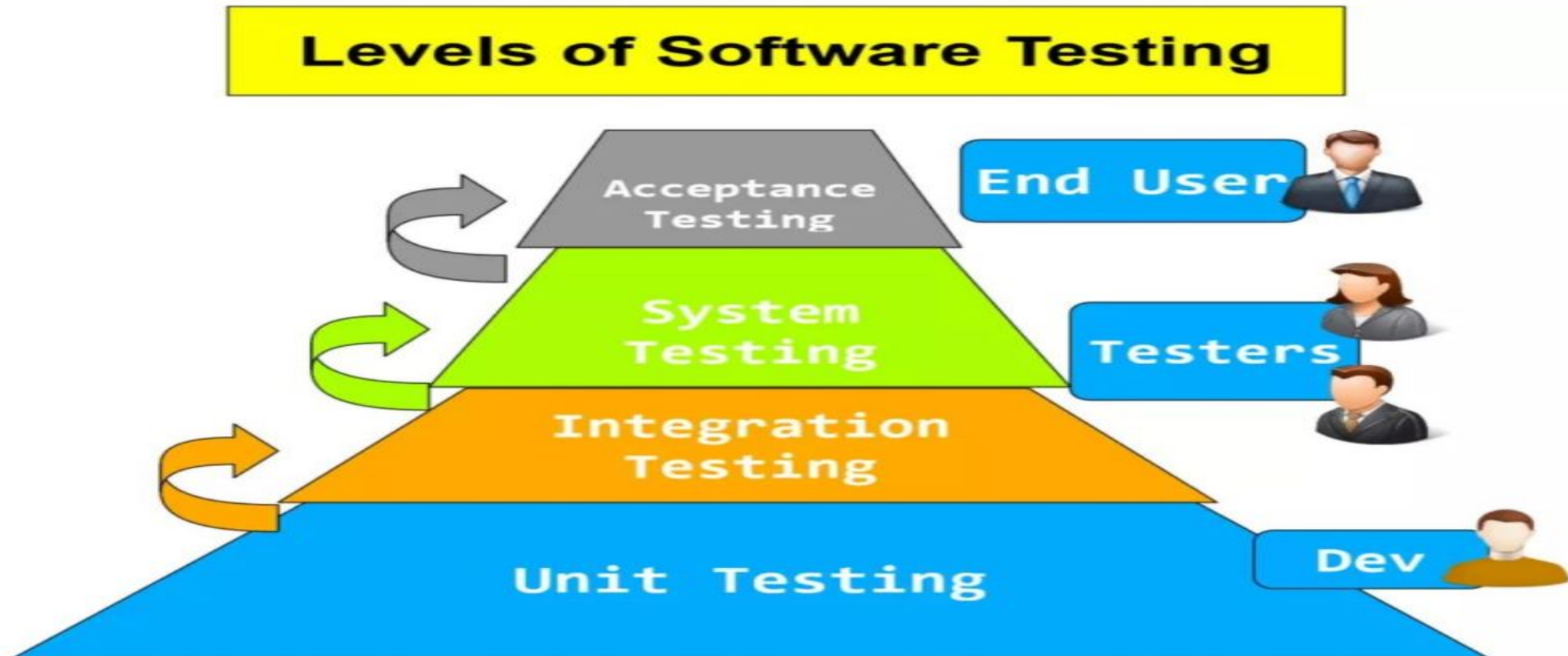
- Automation script automatically tests:
- Login functionality
- API responses
- Regression test suite

Types of Testing: Manual Testing, Automated Testing

Aspect	Manual Testing	Automated Testing
Execution	By human testers	By tools/scripts
Speed	Slow	Fast
Accuracy	Depends on tester	High accuracy
Cost	Low initial cost	High initial setup cost
Best For	Exploratory, UI testing	Regression, load testing
Programming Required	No	Yes
Reusability	Limited	High

Testing Levels

- Testing is performed at different levels during software development to detect errors early and ensure system reliability.



Types of Testing

- **Unit testing** is a method of testing individual units or components of a software application.
- It is typically done by developers and is used to ensure that the individual units of the software are working as intended.
- Unit testing is done at the lowest level of the software development process , where individual units of code are tested in isolation.

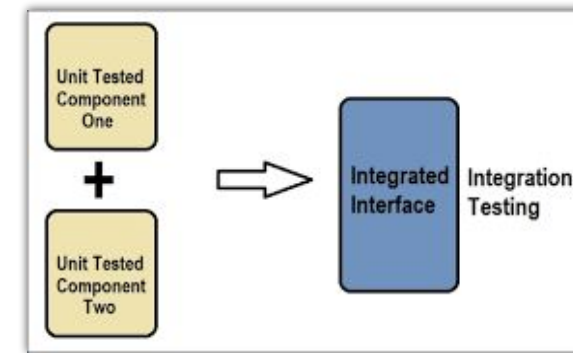
Advantages of Unit Testing:

- Some of the advantages of Unit Testing are listed below.
- It helps to identify bugs early in the development process before they become more difficult and expensive to fix.
- It helps to ensure that changes to the code do not introduce new bugs.
- It makes the code more modular and easier to understand and maintain.
- It helps to improve the overall quality and reliability of the software.

popular frameworks and tools that are used for unit testing include JUnit , NUnit, and xUnit.

Types of Testing

- **Integration testing** is a method of testing how different units or components of a software application interact with each other.
- It is used to identify and resolve any issues that may arise when different units of the software are combined.
- Integration testing is typically done after unit testing and before functional testing and is used to verify that the different units of the software work together as intended.



Different Ways of Performing Integration Testing:

- **Top-down integration testing:** It starts with the highest-level modules and differentiates them from lower-level modules.
- **Bottom-up integration testing:** It starts with the lowest-level modules and integrates them with higher-level modules.
- **Big-Bang integration testing:** It combines all the modules and integrates them all at once
- **Critical Path First**

Top-Down Integration

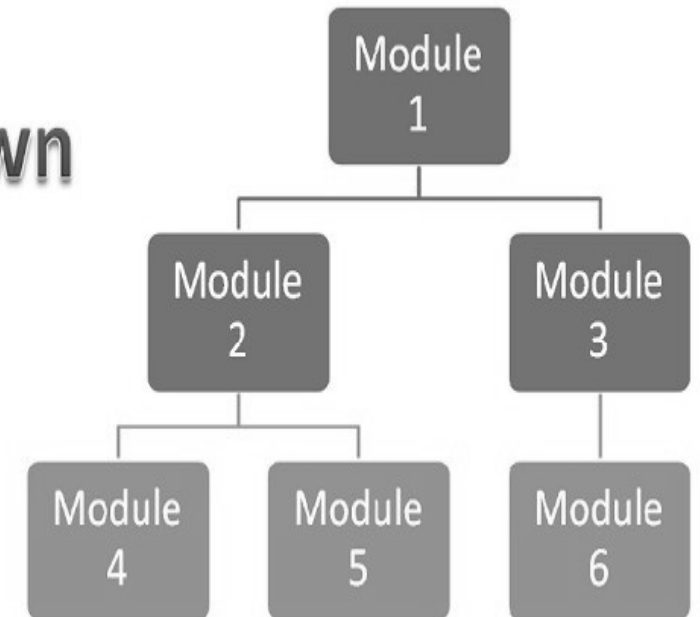
1. The main control module is used as a driver, and stubs are substituted for all modules directly subordinate to the main module.
2. Depending on the integration approach selected (depth or breadth first), subordinate stubs are replaced by modules one at a time.

Approve
leave

Create a
leave Report

Stub

Top Down

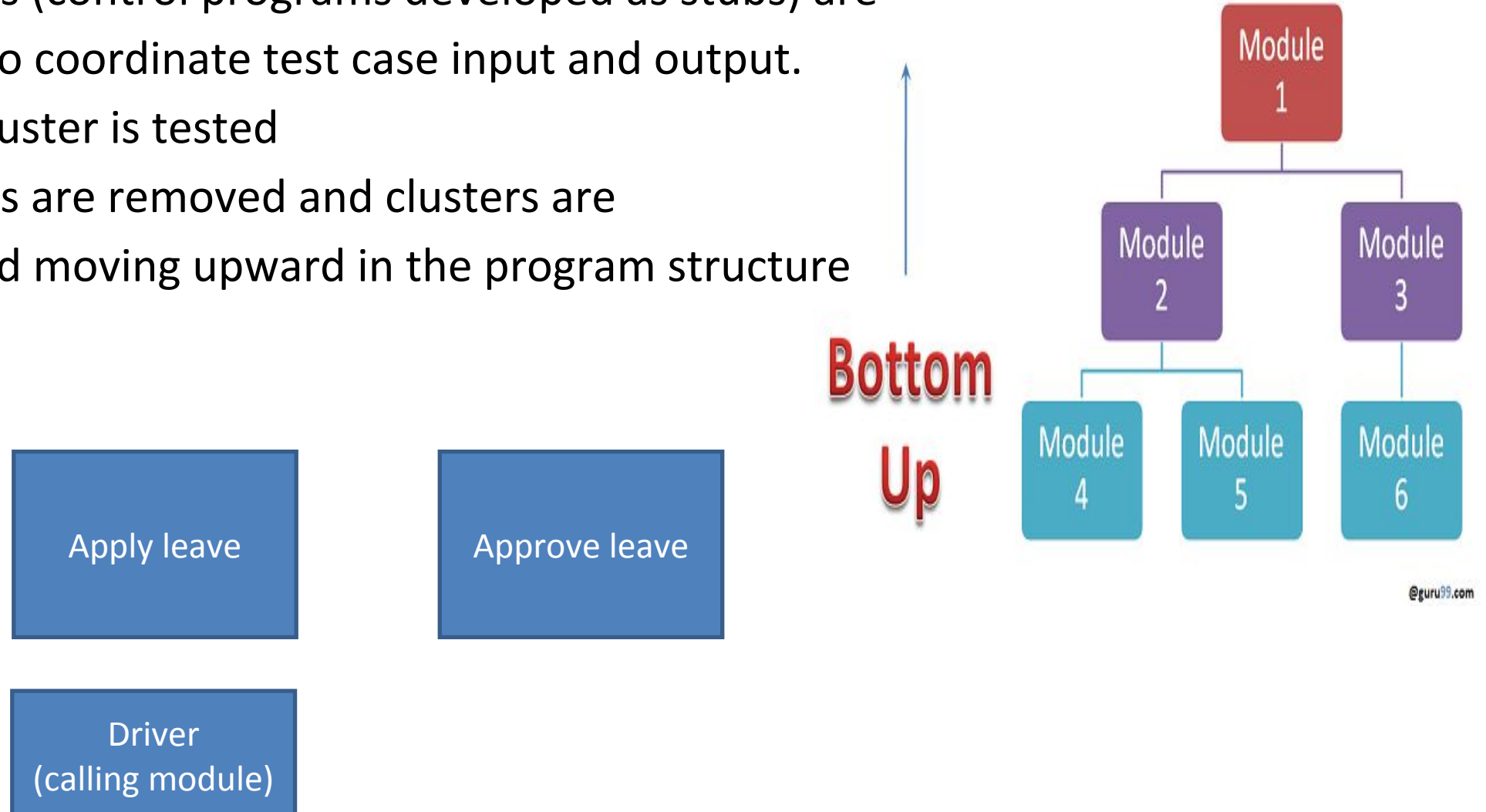


Problems with Top-Down Integration

- Many times, calculations are performed in the modules at the bottom of the hierarchy
- Stubs typically do not pass data up to the higher modules
- Delaying testing until lower-level modules are ready usually results in integrating many modules at the same time rather than one at a time
- Developing stubs that can pass data up is almost as much work as developing the actual module

Bottom-Up Integration

- Integration begins with the lowest-level modules, which are combined into clusters, or builds, that perform a specific software sub function
- Drivers (control programs developed as stubs) are written to coordinate test case input and output.
- The cluster is tested
- Drivers are removed and clusters are combined moving upward in the program structure



Problems with Top-Down Integration

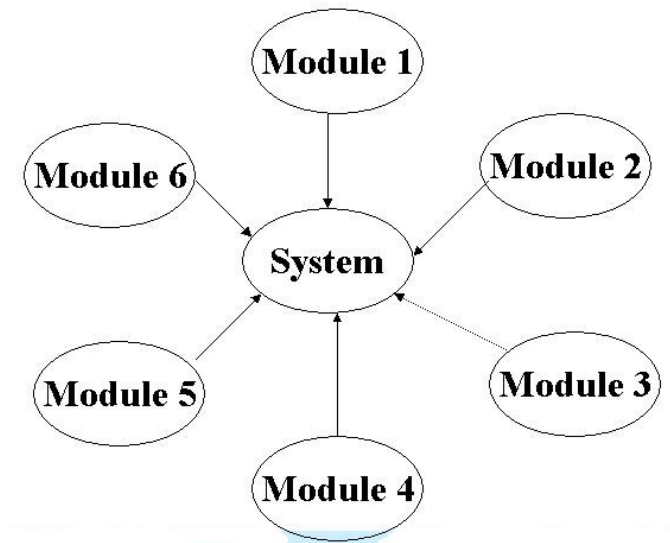
- Delay in Detecting System-Level Issues
- Absence of a Working Prototype Early
- Requires Test Drivers for Each Module
- Difficult to Identify Integration Issues Between High-Level Components
- Increased Cost and Effort

Big Bang Approach in Integration Testing

- Big Bang Integration Testing is an approach where all modules of the system are integrated at once, rather than in phases. After integration, the entire system is tested as a whole.
- This method does not use incremental integration and is typically applied when all components are developed and available simultaneously.

Characteristics of Big Bang Integration Testing

- **All modules are combined** and tested in one go.
- No incremental testing; the system is treated as a **single unit**.
- Requires **all components to be ready** before integration.
- Testing is performed at the **end of the development cycle**.
- Best suited for **small applications** with limited dependencies.



Big Bang Approach in Integration Testing

Advantages of Big Bang Integration Testing:

- **Simple and Time-Saving (Initially)** – Since everything is tested together, there is **no need for test drivers or stubs**.
- **Suitable for Small Systems** – Works well for **small applications** with a limited number of modules.
- **Less Planning Required** – Unlike phased integration, **no complex test schedules** are needed.

Disadvantages of Big Bang Integration Testing

- **High Risk of Failure** – Since all modules are integrated at once, **major system failures** may be detected late.
- **Difficult Debugging** – If the system fails, it is **hard to pinpoint the root cause**, as all modules are tested together.
- **Delayed Testing & Feedback** – Testing occurs **only after all modules are complete**, leading to **late defect detection**.

Advantages of Integrating Testing

- It helps to identify and resolve issues that may arise when different units of the software are combined.
- It helps to ensure that the different units of the software work together as intended.
- It helps to improve the overall reliability and stability of the software.
- It's important to keep in mind that Integration testing is essential for complex systems where different components are integrated.

System Testing

System Testing is a type of software testing that is performed on a completely integrated system to evaluate the compliance of the system with the corresponding requirements.

- In system testing, integration testing passed components are taken as input.
- The goal of integration testing is to detect any irregularity between the units that are integrated.
- It tests if the system meets the specified requirements and if it is suitable for delivery to the end-users.
- This type of testing is performed after the integration testing and before the acceptance testing.

Advantages of System Testing:

- The testers do not require more knowledge of programming to carry out this testing.
- It will test the entire product or software so that we will easily detect the errors or defects that cannot be identified during the unit testing and integration testing.

Acceptance Testing

- It is formal testing according to user needs, requirements, and business processes conducted to determine whether a system satisfies the acceptance criteria or not and to enable the users, customers, or other authorized entities to determine whether to accept the system or not.
- It verifies that the software is **ready for deployment** and satisfies functional and non-functional requirements.

Advantages of Acceptance Testing:

- This testing helps the project team to know the further requirements from the users directly as it involves the users for testing.
- It brings confidence and satisfaction to the clients as they are directly involved in the testing process.
- It is easier for the user to describe their requirement.
- It covers only the Black-Box testing process and hence the entire functionality of the product will be tested.

Types of Acceptance Testing

- **Alpha and Beta Testing** are two types of **Acceptance Testing** performed before software is released to the market. They help identify defects, usability issues, and performance concerns before the final product launch.
- Alpha Testing is performed **in-house** by the **development team, QA team, and sometimes internal users**.
- It occurs **before Beta Testing** and aims to find **bugs, crashes, and usability issues**.
- Conducted in a **controlled environment** (e.g., the company's test lab).

Key Features of Alpha Testing:

- Performed in a **closed environment**.
- Identifies **functional and usability defects** before external users see the software.
- **Simulated real-world conditions** are used for testing.

Example:

A gaming company tests a **new mobile game** internally to detect **game crashes, lags, and bugs** before releasing it for Beta Testing.

Types of Acceptance Testing

- Beta Testing is performed **by real users** in a **real-world environment**.
- The product is released to a **limited number of external users (beta testers)** before the final launch.
- Feedback from beta testers helps in **identifying issues that were missed during Alpha Testing**.

Key Features of Beta Testing:

- Conducted in a **real environment** with **actual users**.
- Helps in collecting **user feedback** and detecting **real-world issues**.
- Bugs found are fixed before the final public release.

Example:

A social media company releases a **beta version of its new messaging app** to a group of users before rolling it out globally.

Types of Acceptance Testing

- Beta Testing is performed **by real users** in a **real-world environment**.
- The product is released to a **limited number of external users (beta testers)** before the final launch.
- Feedback from beta testers helps in **identifying issues that were missed during Alpha Testing**.

Key Features of Beta Testing:

- Conducted in a **real environment** with **actual users**.
- Helps in collecting **user feedback** and detecting **real-world issues**.
- Bugs found are fixed before the final public release.

Example:

A social media company releases a **beta version of its new messaging app** to a group of users before rolling it out globally.

Alpha Vs Beta Testing

Alpha Testing	Beta Testing
Performed at developer's site	Performed at end user's site
Performed in controlled environment as developer is present	Performed in uncontrolled environment as developer is not present
Less probability of finding of errors as it is driven by developers	High probability of finding of errors as end user can use it the way he wants
It is not considered as live application	It is considered as live application
It is done during implementation phase of software	It is done as pre-released of software
Less time consuming as developer can make necessary changes in given time	More time consuming as user has to report bugs id any via appropriate channel

Static vs Dynamic Testing

Static Testing

- This type of testing of software is carried out before putting the software in action. Static testing is carried out to look for the errors in the algorithms, codes or documents.
- This testing is done by the writer or developer of the software or testers and is carried out by walking through it, checking the code reviews, or visual inspection.

Dynamic Testing

- This type of testing is carried out once the software has been fully compiled and loaded to the system.
- In Dynamic testing the software is checked for the consistency of the input and output parameters by using another software.

Static Testing Techniques

Static Testing is the process of evaluating software **without executing the code**.

It is performed to detect defects early by reviewing documents, design, or source code.

- Focus: Error prevention, quality improvement, early defect detection

1. Reviews

Definition

A review is a **systematic examination** of documents or code by one or more people to identify defects and improvements.

Characteristics

- Informal or formal
- No execution of software
- Can include requirement or design reviews

Purpose

- Improve quality
- Detect errors early
- Ensure compliance with standards

Example

Reviewing an SRS document before development starts.

Static Testing Techniques

Walkthrough: A walkthrough is a **semi-formal review** where the author explains the document or code step-by-step to a group.

Characteristics

- Led by the author
- Interactive discussion
- Focus on understanding logic

Purpose

- Knowledge sharing
- Early defect identification
- Improve design clarity

Example

Developer explains program flow to team members

Static Testing Techniques

- **Inspections:** Inspection is a **formal and structured review process** performed by a trained team to detect defects.

Characteristics

- Highly structured process
- Defined roles (moderator, reader, recorder)
- Uses checklists
- Detailed documentation

Purpose

- Find defects systematically
- Ensure high-quality standards

Example

- Formal code inspection before release.

Static Testing Techniques

Technique	Formality	Led By	Focus
Review	Informal/Formal	Team	General quality check
Walkthrough	Semi-formal	Author	Understanding & discussion
Inspection	Highly formal	Moderator	Detailed defect detection

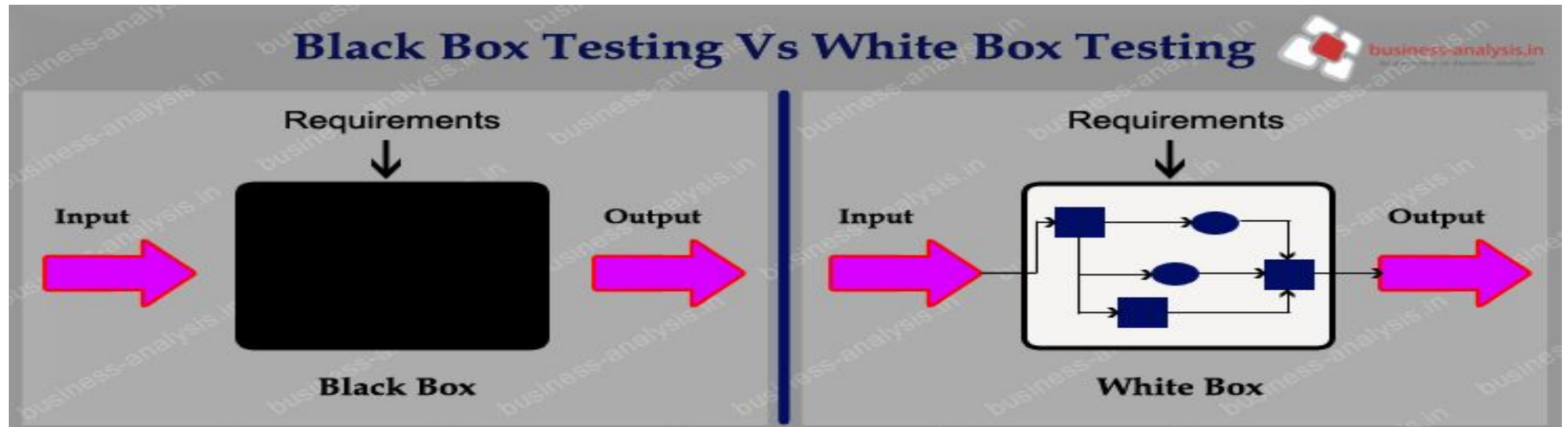
Testing Types

Static

Dynamic

Black Box

White Box

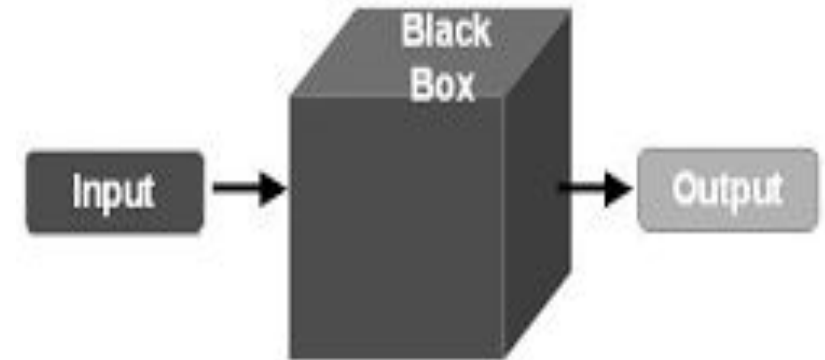


Black Box , White Box Testing

- In the **Software Testing** field, various methods are used to find defects, which used to increasing the software's quality.
- Black-Box Testing is a Type of Software Testing in which the tester is not concerned with the software's internal knowledge or implementation details but rather focuses on validating the functionality based on the provided specifications or requirements.
- White-box Testing is a Software Testing Technique that involves testing the internal structure and workings of a Software Application. The tester has access to the source code and uses this knowledge to design test cases that can verify the correctness of the software at the code level.
- It is also called **Glass Box Testing** or **Clear Box Testing** or **Structural Testing**.

Types of Dynamic Testing

Black-box testing is a type of software testing in which the tester is not concerned with the internal knowledge or implementation details of the software but rather focuses on validating the functionality based on the provided specifications or requirements.



Advantages of Black Box Testing:

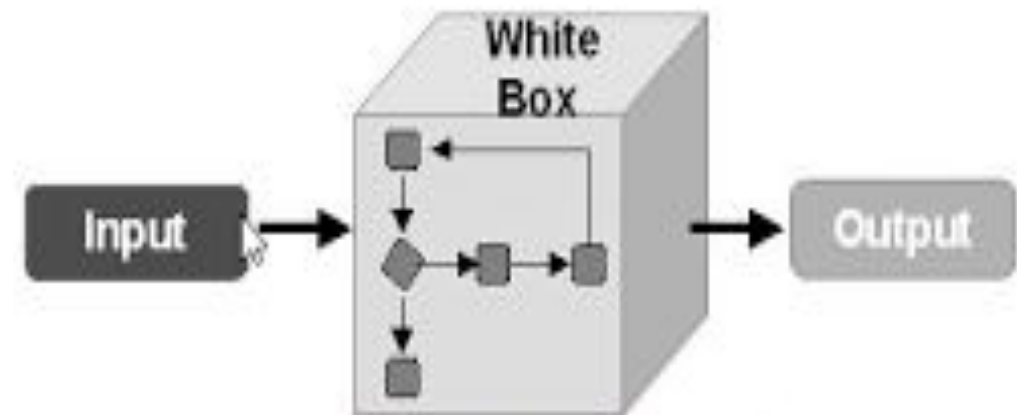
- The tester does not need to have more functional knowledge or programming skills to implement the Black Box Testing.
- It is efficient for implementing the tests in the larger system.
- Tests are executed from the user's or client's point of view.
- Test cases are easily reproducible.
- It is used to find the ambiguity and contradictions in the functional specifications.

Types of Dynamic Testing

- **White box testing** techniques analyze the internal structures the used data structures, internal design, code structure, and the working of the software rather than just the functionality as in black box testing.
- It is also called glass box testing clear box testing or structural testing.
- White Box Testing is also known as transparent testing or open box testing.
- White box testing is a software testing technique that involves testing the internal structure and workings of a software application.
- The tester has access to the source code and uses this knowledge to design test cases that can verify the correctness of the software at the code level.

Advantages of White box Testing:

- Thorough Testing
- Early Detection of Defects
- Integration with SDLC
- Detection of Complex Defects

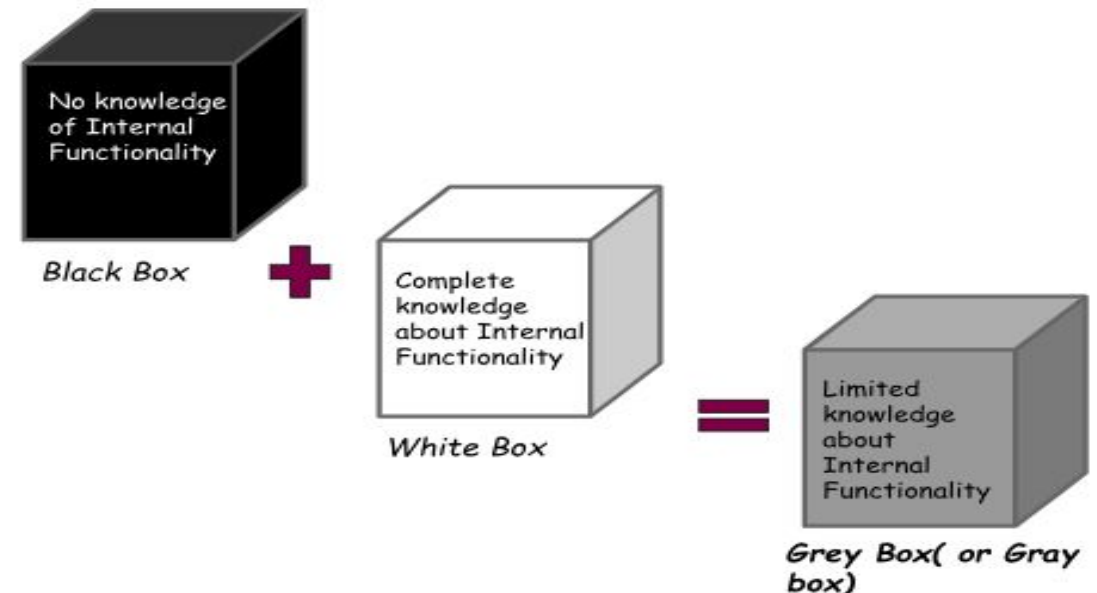


Types of Dynamic Testing

- **Gray Box Testing** is a software testing technique that is a combination of the Black Box Testing technique and the White Box Testing technique.
- In the Black Box Testing technique, the tester is unaware of the internal structure of the item being tested and in White Box Testing the internal structure is known to the tester.
- The internal structure is partially known in Gray Box Testing.
- This includes access to internal data structures and algorithms to design the test cases.

Advantages of Gray Box Testing:

- Clarity of goals
- Done from a user perspective
- High programming skills not required
- Improved product quality



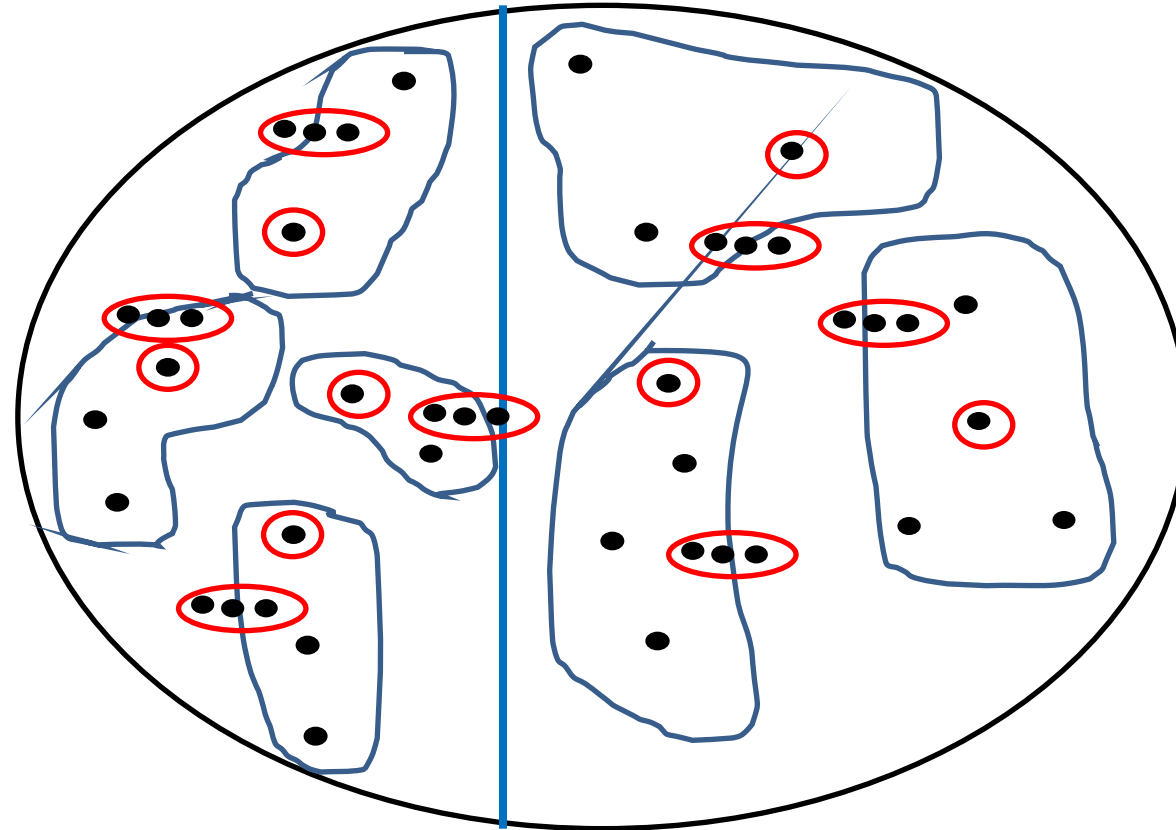
Types of Black-Box Testing

- **Positive/Negative-** checks both good/bad results
- **Boundary value analysis**
- **Decision tables**
- **Equivalence partitioning-** group related inputs/outputs

Boundary Value Analysis

- Boundary value analysis is done to check **boundary conditions**.
- A boundary value analysis is a testing technique in which the elements at the edge of the domain are selected and tested.
- Using boundary value analysis, instead of focusing on input conditions only, the test cases from output domain are also derived.
- Boundary value analysis is a test case design technique that complements equivalence partitioning technique.

 Create test cases to test boundaries of equivalence classes



Decision tables

Decision Table for `daysInMonth(month, year)` Conditions

1. Month in {1,3,5,7,8,10,12}?
2. Month in {4,6,9,11}?
3. Month == 2?
4. Leap Year? (True/False)
5. Invalid Month?

Actions

- Return 31
- Return 30
- Return 29
- Return 28
- Return -1

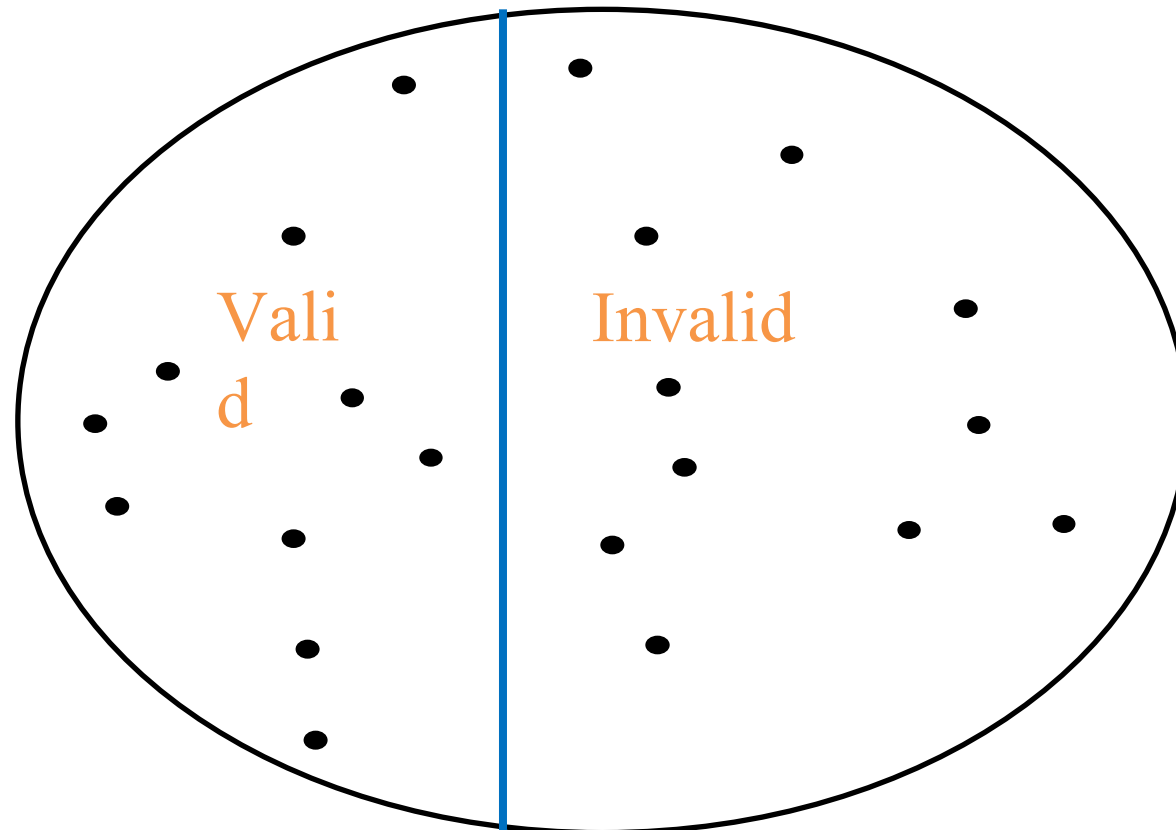
Equivalence testing

- **Equivalence Partitioning:**

- A black-box test technique to reduce no of required test cases.
- steps in equivalence testing:
 - **identify classes of inputs with same behavior**
 - **test on at least one member of each equivalence class**
 - **assume behavior will be same for all members of class**
- criteria for selecting equivalence classes:
 - *coverage* : every input is in one class
 - *disjointedness* : no input in more than one class
 - *representation* : if error with 1 member of class, will occur with all

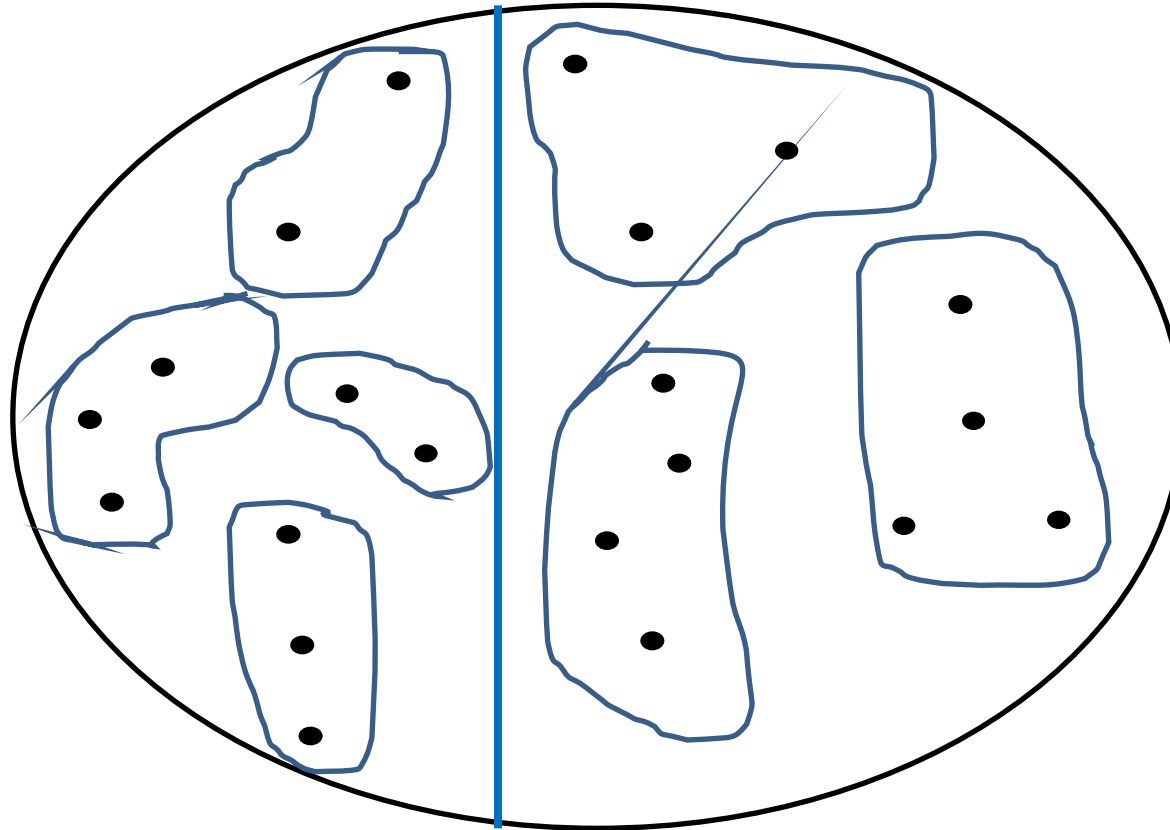
Equivalence Partitioning

- First-level partitioning: Valid vs. Invalid test cases



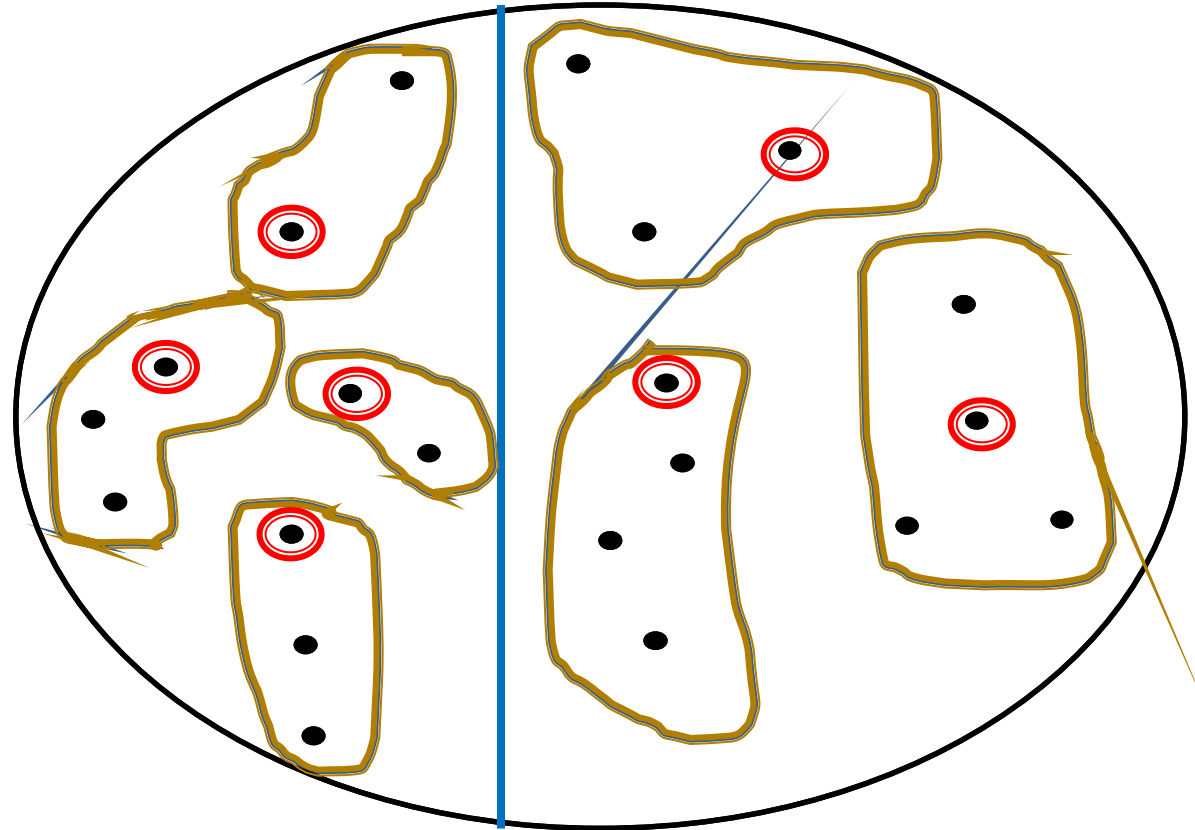
Equivalence Partitioning

Partition valid and invalid test cases into equivalence classes



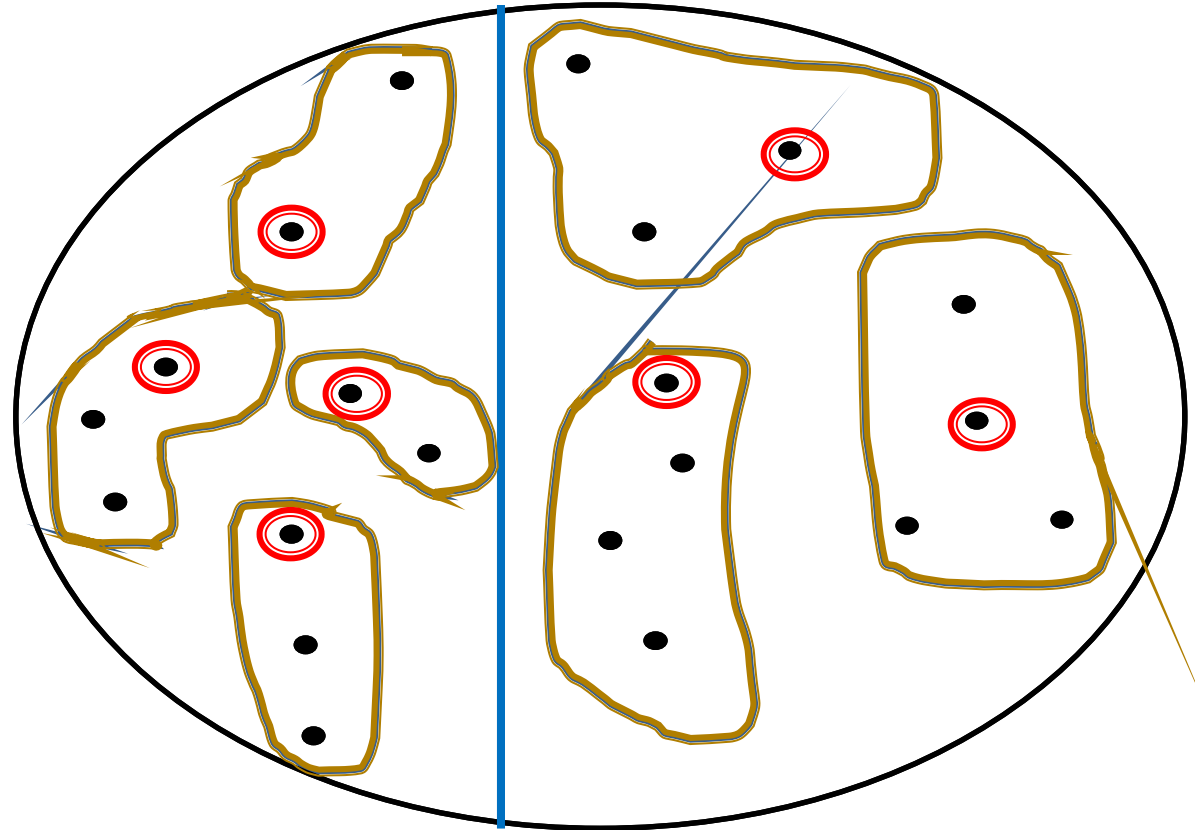
Equivalence Partitioning

- Create a test case for at least one value from each equivalence class



Equivalence Partitioning

- Create a test case for at least one value from each equivalence class



Static (GAP Analysis) vs. Dynamic Testing

Static Testing (GAP Analysis)

- **Definition:** Testing software *without executing the code*. It focuses on finding defects in requirements, design, and documentation.
- **Objective:** Detect defects early, before coding starts.
- **Examples:**
 - Reviews (requirement, design, code reviews)
 - Walkthroughs, Inspections
 - Static code analysis
 - **GAP Analysis:** Comparing *as-is* (current state) with *to-be* (desired state) requirements to identify missing functionalities or mismatches before development/testing.

Different types of White Box Testing

- **Unit Testing**: Tests individual functions or components in isolation to ensure they work correctly.
- **Static Code Analysis**: Analyzes code without executing it to find syntax errors, code smells, or potential bugs.
- **Dynamic Testing**: Examines the code while it runs to evaluate behavior, performance, and memory usage.
- **Statement Coverage**: Ensures every line of code is executed at least once during testing.
- **Branch Testing**: Verifies that all possible branches (if/else conditions) in the code are tested.
- **Path Testing**: Checks that all possible logical paths through the code are executed.
- **Loop Testing**: Tests loops in the code with different iteration conditions to catch errors in looping logic.

White-box testing

- **Path testing:** an attempt to use test input that will pass once over each path in the code
 - path testing is *white* box
 - What would be path testing for `daysInMonth(month, year)`?

```
def daysInMonth(month, year):  
    if month in {1, 3, 5, 7, 8, 10, 12}:  
        return 31  
    elif month in {4, 6, 9, 11}:  
        return 30  
    elif month == 2:  
        if (year % 400 == 0) or ((year % 4 == 0) and (year % 100 != 0)):  
            return 29 # leap year  
        else:  
            return 28  
    else:  
        return -1 # invalid month  
        – in a leap year, not in a leap year
```

Year is **divisible by 400** → always a leap year
(e.g., 1600, 2000, 2400).
Year is **divisible by 4 but not by 100** → leap year
(e.g., 1996, 2004, 2020).
If neither condition is true → **not a leap year**
(e.g., 1900, 2100, 2021).

Thank you!

